Product Requirements Document: Kortex

Agentic Knowledge Copilot by Neural Orchestration Core (NOC)

Status: Draft / Hackathon Ready
Lead Architect: charan sai Ponnada
Last Updated: Apr 6, 2026

1. Vision

Kortex is an agentic enterprise knowledge copilot designed to transform fragmented organizational data into actionable intelligence. It enables engineers to retrieve, reason over, and act on knowledge across documents and historical tickets through a multi-agent orchestration framework.

2. Problem Statement

Enterprise knowledge is often scattered across disparate sources like PDFs, SOPs, internal wikis, and support tickets, leading to significant operational friction:

- **Slow onboarding:** New engineers struggle to find relevant documentation.
- **Repetitive queries:** Support teams answer the same technical questions repeatedly.
- **Knowledge silos:** Critical information is isolated within specific teams.
- **Inefficient workflows:** Debugging is slowed by the lack of unified historical context.

Existing RAG systems often fail because they lack autonomous decision-making, validation, and multi-source reasoning, resulting in unreliable or shallow outputs.

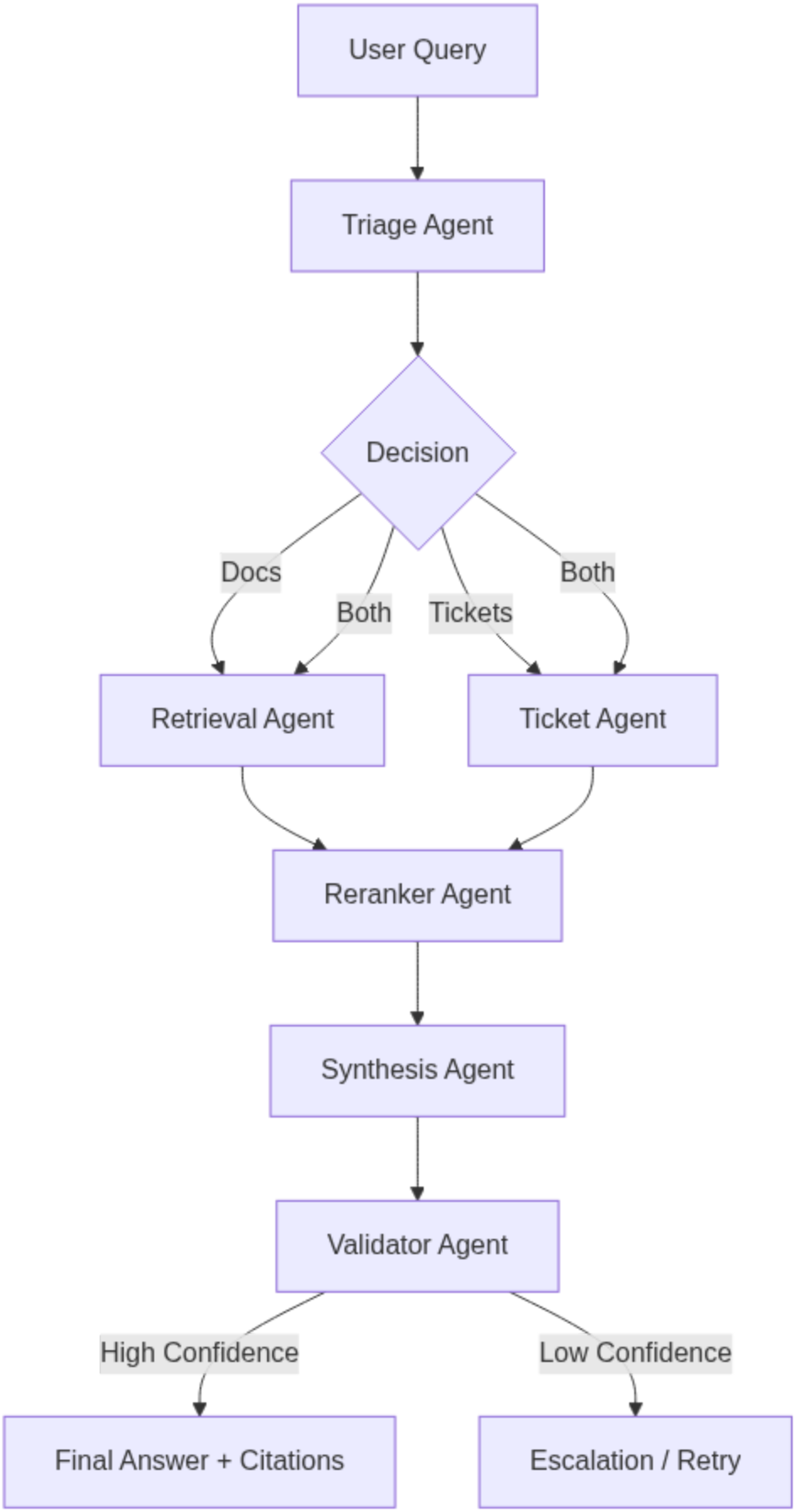
3. Solution Overview

Kortex introduces a multi-agent RAG system where specialized agents collaboratively retrieve, validate, and synthesize knowledge. Unlike traditional linear pipelines, Kortex dynamically orchestrates tool usage and applies confidence-aware validation to ensure grounded, reliable responses.

4. System Architecture

High-Level Flow

Kortex utilizes a non

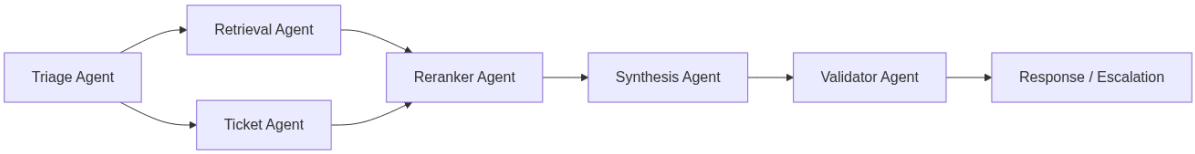


-linear flow to

handle complex queries efficiently

Agent Map (Core Intelligence Layer)

The intelligence layer is divided into specialized modules that interact via a centralized orchestration framework:



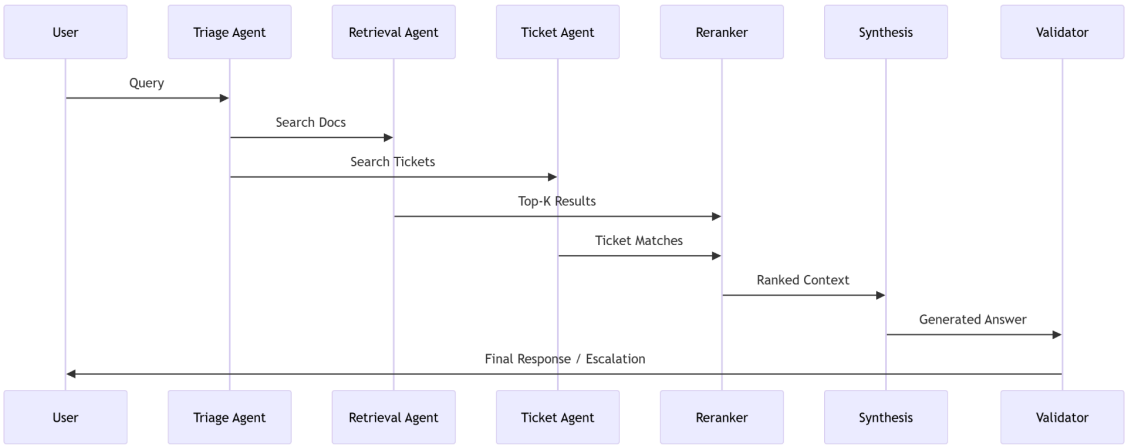
5. Component Breakdown

Component	Responsibility	Key Functionality
5.1 Triage Agent	Intent Interpretation	Determines query type and selects tools dynamically.
5.2 Retrieval Agent	Semantic Search	Performs top-K document chunk retrieval from vector DB.
5.3 Ticket Agent	Historical Analysis	Retrieves real-world resolution patterns from ticket DB.
5.4 Reranker Agent	Precision Tuning	Uses cross-encoder models to improve result relevance.
5.5 Synthesis Agent	Answer Generation	Combines multi-source inputs into a structured response.

Component	Responsibility	Key Functionality
5.6 Validator Agent	Quality Control	Detects hallucinations and computes confidence scores.

6. Detailed Workflow

The interaction follows a structured sequence to ensure data integrity:



1. **User** submits a query to the **Triage Agent**.
2. **Triage Agent** routes the request to **Retrieval** (Docs) and **Ticket** agents.
3. **Reranker Agent** optimizes the context for the **Synthesis Agent**.
4. **Synthesis Agent** generates a response which is then audited by the **Validator Agent**.
5. Final response is delivered or the system triggers an escalation protocol.

7. Key Features

- **Multi-agent orchestration:** Utilizes ReAct or Plan-Execute patterns.
- **Hybrid retrieval:** Simultaneously queries technical documentation and historical tickets.
- **Reranking:** Enhances precision using advanced cross-encoder models.
- **Confidence-aware validation:** Proactive hallucination detection and escalation.

- **Citation-backed responses:** Every answer points directly to its source.

8. Evaluation Metrics

Success will be measured using the following quantitative and qualitative KPIs:

- **Retrieval Quality:** Precision@K and Recall@K.
- **Accuracy:** Semantic Similarity Score and LLM-as-Judge Evaluation.
- **Performance:** Latency and response time metrics.
- **Reliability:** Hallucination rate and user satisfaction demo feedback.

9. Tech Stack

Layer	Technology
Backend	FastAPI
AI/ML	Mistral / LLaMA, Sentence Transformers, Cross-Encoder
Data	FAISS / Qdrant (Vector DB)
Orchestration	LangGraph / Custom ReAct loop
Frontend	Streamlit / React

10. Guardrails & Responsible AI

- **Confidence Filtering:** Responses below a threshold are never shown to users.
- **Source Grounding:** No generation is allowed without direct context mapping.
- **Privacy:** Optional PII redaction layer for sensitive ticket data.
- **Escalation:** Human-in-the-loop triggers for low-confidence scenarios.

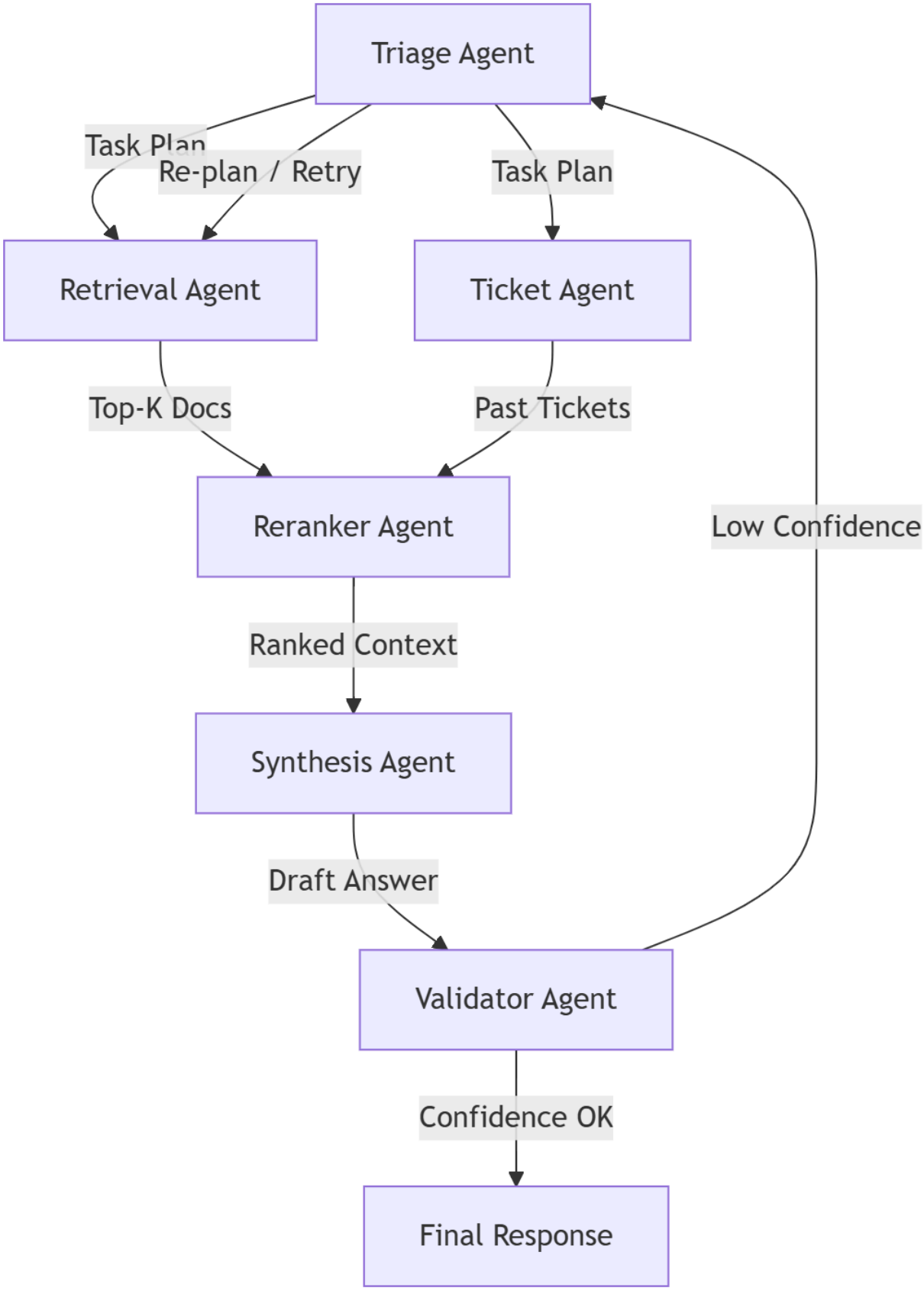
11. Risks & Mitigation

Risk	Mitigation Strategy
Poor retrieval quality	Implement better chunking strategies and reranking.
Hallucination	Enforce strict grounding and secondary validation agents.
High Latency	Limit agent loops and optimize model inference.
Over-complexity	Maintain a modular, extensible architecture.

12. Future Scope

- **Knowledge Graph:** Integration for deeper relationship reasoning.
- **Multimodal:** Support for images and tables via LayoutLM.
- **Self-Improvement:** Automated feedback loops based on user corrections.
- **Enterprise APIs:** Deployment-ready endpoints for third-party integrations.

13. Agent-to-Agent Orchestration


Triage Agent
Task Plan
Re-plan / Retry
Task Plan
Retrieval Agent
Ticket Agent
Top-K Docs
Past Tickets
Reranker Agent
Low Confidence
Ranked Context
Synthesis Agent
Draft Answer
Validator Agent
Confidence OK
Final Response

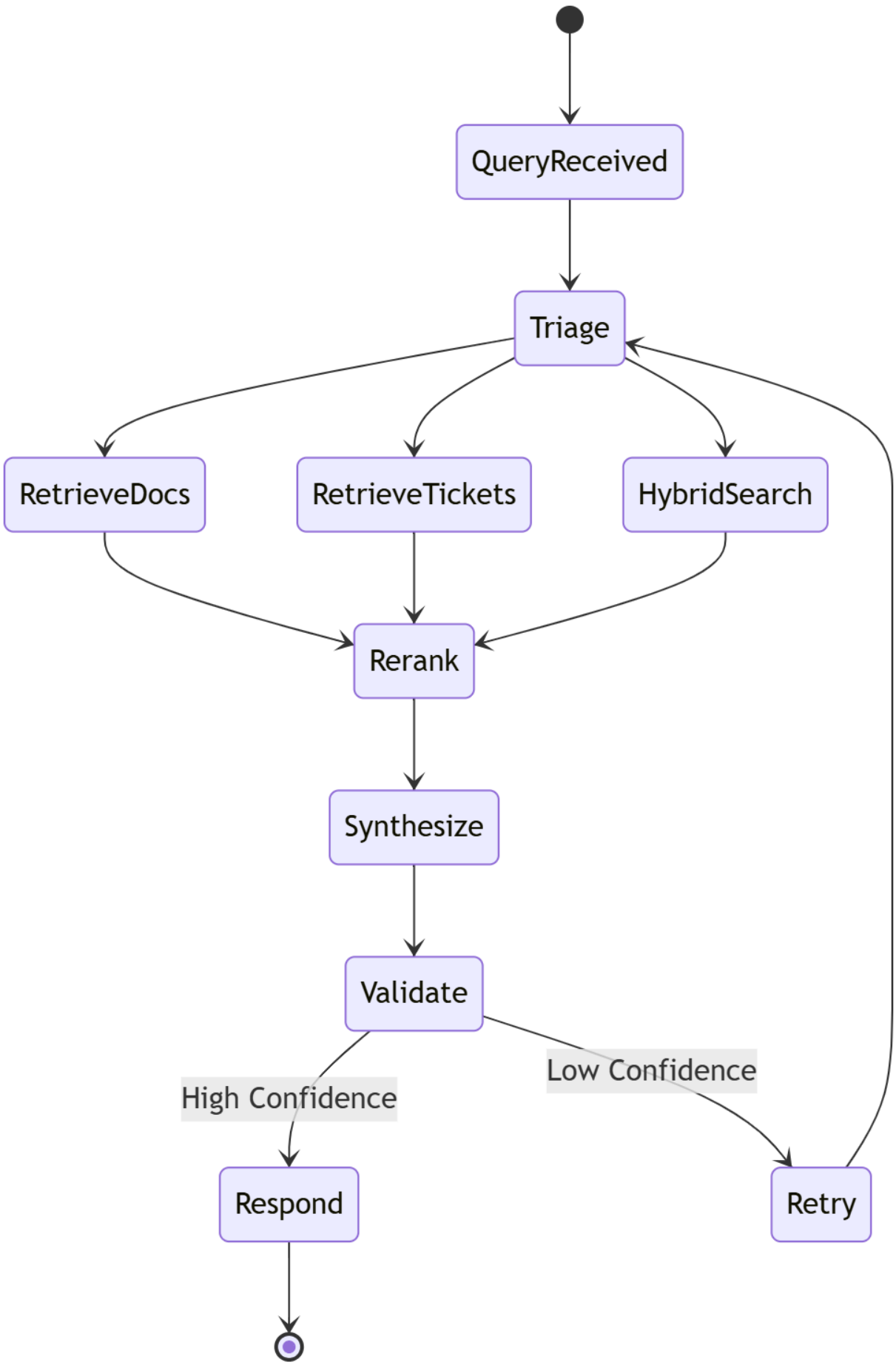
Kortex employs a structured Agent-to-Agent (A2A) communication model where specialized agents collaborate through task delegation and iterative refinement.

- The Triage Agent acts as the planner, delegating tasks to retrieval and ticket agents.
- Intermediate outputs are passed through a reranking layer before synthesis.
- The Validator Agent enforces reliability by either approving responses or triggering re-execution cycles.
- This loop enables adaptive reasoning, making the system dynamic rather than static.

This architecture follows a Plan-Execute-Validate paradigm, enabling intelligent orchestration across multiple agents.

14. Agent State Machine (System Intelligence)

This diagram illustrates the stateful and adaptive nature of the Kortex system, moving beyond a simple linear pipeline by incorporating decision-based transitions and intelligent looping.


QueryReceived
Triage
RetrieveDocs
RetrieveTickets
HybridSearch
Rerank
Synthesize
Validate
High Confidence
Low Confidence
Respond
Retry

Why this is powerful

- Shows looping intelligence
- Shows decision-based transitions
- Matches real systems like LangGraph / agent frameworks

15. Confidence Scoring Mechanism (Elite Depth)

To ensure reliable output, Kortex uses a quantitative confidence score, providing engineering maturity and hitting judging criteria for reliability.

Formula

Confidence Score = (0.5 × Retrieval Similarity) + (0.3 × Reranker Score) + (0.2 × LLM Self-Evaluation)

Breakdown

- **Retrieval Similarity:** Calculated using cosine similarity from the vector database (vector DB).
- **Reranker Score:** Derived from the cross-encoder model's relevance score.
- **LLM Self-Eval:** Generated via a prompt to the Synthesis LLM: “Rate confidence from 0–1 based on context completeness.”

Decision Logic

- If Confidence > 0.75 → Return Answer
- If 0.5–0.75 → Retry (triggers search expansion or query refinement)
- If < 0.5 → Escalate (for human-in-the-loop review)

17. API Design (Production-Level Thinking)

Kortex exposes a production-ready API endpoint for seamless integration into enterprise systems.

🧾 **Query Endpoint**

POST /query

Request

```JSON
{
  "query": "How to fix Kafka consumer lag?",
  "user_id": "123",
  "context_mode": "auto"
```

```
}
```

Response

```JSON
{
  "answer": "To resolve Kafka consumer lag...",
  "sources": [
    { "doc": "Kafka Guide.pdf", "page": 12 },
    { "ticket": "INC-1023" }
  ],
  "confidence": 0.87,
  "agent_trace": [
    "Triage → Docs + Tickets",
    "Reranked top 3 results",
    "Validated response"
  ],
  "status": "success"
}
```

⚠ Escalation Response

```JSON
{
  "status": "escalated",
  "reason": "Low confidence",
  "suggestion": "Forward to human engineer"
}
```

18. Agent Trace (Explainability)

This can be the Game Changer (We are literally showing why the agent is taking the file or decision).

Kortex maintains an agent trace log that records each decision and action taken by the system. This enables transparency, debugging, and explainability, allowing users to understand how the system arrived at a particular answer.

19.System Design Philosophy (Elite Positioning)

Kortex is designed as a cognitive system rather than a static pipeline. It incorporates planning, execution, validation, and adaptive retry mechanisms, enabling it to behave as an intelligent collaborator rather than a passive responder.

20. UI Wireframe (Judge-Winning Layout)

Kortex's interface is designed to showcase its agentic intelligence in real-time, providing transparency and proof of the multi-agent orchestration to the judges.

🖥️ Main Screen Layout

```None
+--------------------------------------------------+
| 🔷 Kortex — Agentic Knowledge Copilot             |
+--------------------------------------------------+
| 🔍 Query Input                                    |
| "How to fix Kafka consumer lag?"                  |
| [Ask Kortex]                                      |
+--------------------------------------------------+
| 🧠 Agent Activity Panel (LEFT)   | 📄 Response (RIGHT)
|
|--------------------------------|------------------------
-|
| Triage: Analyzing intent...    | Answer:
|
| Retrieval: Searching docs...   | Step 1: ...
|
```

```
| Ticket: Checking past issues   | Step 2: ...
|
| Reranker: Filtering results    |
|
| Validator: Confidence 0.87     | Sources:
|
|                                | - Kafka.pdf (p12)
|
|                                | - Ticket #1023
|
|                                |
|
|                                | Confidence: 87%
|
+--------------------------------------------------+
| ⚠️ Escalation (if low confidence)                 |
| "Not confident. Escalating to human."             |
+--------------------------------------------------+
```

🔥 What Makes This UI Powerful

- **Left Panel = "AI Thinking":** Shows the entire agent flow (Triage, Retrieval, Reranking, Validation), allowing judges to visibly track the multi-agent system working.
- **Right Panel = Clean Output:** Delivers clear, immediate value (Answer, Sources, Confidence).
- **Bottom = Escalation:** Demonstrates real-world reliability and robustness, proving the system handles low-confidence scenarios responsibly.

🎯 Optional (High Impact)

- Progress animation (typing/loading)
- Step-by-step reveal (like ChatGPT reasoning)

21. Execution & Delivery Plan (The Final Sprint)

This plan ruthlessly prioritizes a working, high-impact demo over non-essential features, leveraging our team's specialized skills.

👥 Team Role Breakdown (CRITICAL)

Team Member	Responsibility (The Brain)	Tasks (The Hands)	Status
Charan Sai Ponnada (AI & System Architect)	**Agent Flow Design, LLM Engineering, and Integration**	Implement: Triage logic, Agent orchestration, Validator, Integrate everything	☐
Oohaa	**Backend, Data, and Retrieval Engine**	Implement: Chunking + embeddings, FAISS / Qdrant setup, Cross-encoder reranker, Ticket dataset integration	☐
Neelima	**Frontend, UX, and Demo Visualization**	Build UI (Streamlit / React), Agent activity panel, Response display, Demo polish	☐

Team Workflow (IMPORTANT)

A linear, dependency-aware workflow:
Charan (logic) → Oohaa (data/retrieval) → Charan (integration) → Neelima (UI + demo)

Rules You MUST Follow

- **Don't:** All 3 people doing same thing; Overcomplicate frontend; Waste time on styling.
- **Do:** Focus on working pipeline first; Then UI polish; Then demo storytelling.
- **Brutal Truth:** UI wins attention, Agents win judges, Clarity wins hackathon.

⏱️ Suggested Timeline (Tight Execution)

Day	Focus	Deliverables
Day 1	**Core Backend & RAG**	Core RAG pipeline working (Retrieval + Synthesis). Basic agent flow implemented.
Day 2	**Agent Intelligence & Validation**	Add reranking + validator. Full agent orchestration (A2A) integrated. UI integration begins.
Day 3	**Demo Polish & Edge Cases**	Final UI polish. Agent Activity Panel fully functional. Test and showcase edge cases (especially the low-confidence Escalation loop).